SISTEMI AD AGENTI · A.A. 2025/26

# Chatbot clinico per la stima del rischio

Un agente conversazionale che raccoglie dati clinici attraverso un dialogo guidato e li trasforma in una stima di rischio, usando embedding semantici biomedici e un classificatore addestrato su dati reali.

❤️ Heart Disease | 🏥 Diabetes Readmission | BioBERT · 768-d | Streamlit · Telegram

Corso di **Sistemi ad Agenti** Docente: **Prof.ssa Berardina De Carolis** **Università degli Studi di Bari "Aldo Moro"**

**Note:** Apertura: presento il progetto in una frase — un chatbot che fa domande cliniche e restituisce una stima di rischio. Sottolineo subito i due aspetti che l'esame richiede: è un *agente* (percepisce, mantiene uno stato, agisce) e la sua decisione finale non è una regola scritta a mano ma un modello appreso dai dati.

PERCORSO

# Di cosa parliamo


| | | |
|---|---|---|
| 01 | **Cos'è un chatbot, e perché è un agente** | slide 3–5 |
| 02 | **Dal dialogo alla predizione** | slide 6–9 |
| 03 | **Classificazione e apprendimento** | slide 10–13 |
| 04 | **I due dataset e come scegliere** | slide 14–16 |
| 05 | **Come si esegue il progetto** | slide 17–18 |
| 06 | **Librerie, limiti e sviluppi** | slide 19–20 |


**Note:** Dichiaro il percorso: si parte dalla teoria (cos'è un chatbot, come si inquadra nella teoria degli agenti), si scende nel funzionamento concreto, poi i dati, infine l'esecuzione pratica. Chiudo con limiti e sviluppi, che è la parte che mostra consapevolezza critica.

# Cos'è un chatbot

Un sistema software che interagisce con l'utente in **linguaggio naturale**, attraverso turni di dialogo, per fornire informazioni o portare a termine un compito.

ASSE 1 · SCOPO

**Task-oriented vs open-domain**

Obiettivo preciso in un dominio chiuso (prenotare, diagnosticare) oppure conversazione libera senza compito.

ASSE 2 · RISPOSTA

**Regole · retrieval · generativo**

Risposte scritte a priori, recuperate da una base di conoscenza, o generate da un modello linguistico.

ASSE 3 · DIALOGO

**Slot-filling · frame · piani**

Riempimento sequenziale di campi, gestione a frame con più intenti, o pianificazione dinamica dei turni.

**Dove si colloca il nostro sistema**

**Task-oriented** (un solo compito: stimare un rischio clinico) · dialogo **a regole con slot-filling deterministico** · ma la conclusione non è una regola: è la previsione di un **modello statistico appreso dai dati**. È un ibrido — regole per condurre la conversazione, machine learning per decidere.

**Note:** Definisco il chatbot e lo classifico su tre assi, perché all'esame conta saper collocare il sistema, non solo descriverlo. Il punto chiave da far passare: il nostro dialogo è deterministico e prevedibile (nessun LLM che genera testo libero), mentre la parte "intelligente" è concentrata nella decisione finale, che è appresa. Questa separazione è una scelta progettuale: rende il comportamento verificabile e la previsione misurabile.

# Il chatbot come agente: descrizione PEAS

**P · PERFORMANCE**

Qualità della stima (ROC-AUC, macro-F1), questionario portato a termine, robustezza agli input non validi, trasparenza dell'esito (probabilità + soglia + avvertenza).

**E · ENVIRONMENT**

Conversazione testuale con un utente umano via Streamlit o Telegram. **Parzialmente osservabile** (vede solo ciò che l'utente dichiara), **episodico** (una sessione = una valutazione), deterministico, statico, discreto.

**A · ACTUATORS**

Messaggi di testo: la domanda successiva, un messaggio d'errore mirato, oppure l'esito finale con probabilità stimata e disclaimer.

**S · SENSORS**

I messaggi di testo dell'utente, interpretati da **parser specifici per tipo di slot** (scelta, intero in intervallo, decimale, codice etichettato).

### Proprietà dell'agente

**Autonomia**: decide da sé quale domanda porre e quando concludere · **Reattività**: risponde a ogni turno dell'utente · **Orientamento all'obiettivo**: completare gli slot e produrre la stima · **Persistenza dello stato**: ricorda le risposte già date nella sessione.

**Note:** Questa è la slide che lega il progetto al programma del corso. Mi soffermo su "parzialmente osservabile": l'agente non misura nulla, conosce solo ciò che l'utente gli dichiara — il che è anche il suo limite principale, perché l'utente può sbagliare o non conoscere il valore. "Episodico" significa che ogni sessione è indipendente: non c'è memoria tra una valutazione e l'altra.

# Architettura dell'agente

**Sensore — messaggio dell'utente**

`handle_message(session, text)` riceve il turno

↓

**Interpretazione — parser dello slot corrente**

Validazione del valore; se fallisce, l'errore non fa avanzare lo stato

↓

**Stato interno — `ConversationSession`**

Risposte raccolte ( `answers` ) + indice della domanda ( `step` )

↓

**Decisione appresa — solo a slot completi**

`record_to_text → BioBERT → LogisticRegression → soglia τ`

↓

**Attuatore — risposta testuale**

Domanda successiva · messaggio d'errore · esito finale

### Che tipo di agente è

Un **agente basato su modello, con stato interno**: non è puramente reattivo, perché la risposta dipende da quanto già raccolto nella sessione, non solo dall'ultimo messaggio.

### Dove sta l'apprendimento

La componente decisionale è **appresa offline**, prima che la conversazione inizi. Durante il dialogo l'agente **non apprende**: applica un modello già addestrato e congelato.

### Un solo cervello, due canali

Streamlit e Telegram chiamano lo stesso `chatbot_core` : la logica dell'agente è unica, cambia soltanto il mezzo di comunicazione.

---

**Note:** Mostro il ciclo percepisci–aggiorna stato–agisci. Faccio notare che il modulo decisionale si attiva una sola volta per sessione, all'ultimo slot: fino a quel momento l'agente è puramente dialogico. E chiarisco bene la distinzione tra apprendimento offline (addestramento) e inferenza online (uso): l'agente non impara dalle risposte degli utenti.

# Il dialogo: slot-filling guidato

Ogni dataset ha una **lista ordinata di domande**, una per feature. Ciascuna definisce quattro cose:

`field` — il campo da riempire
`prompt` — la domanda da porre
`parser` — come interpretare e validare
`error` — cosa dire se il valore non è valido

### La regola di avanzamento

Se il parser restituisce un valore valido → salva e passa alla domanda dopo. Se restituisce `None` → ripete l'errore **senza avanzare**. L'agente non accetta mai un valore che non sa interpretare.

**13**
DOMANDE · HEART

**19**
DOMANDE · DIABETES

CONVERSAZIONE REALE

Ciao! Sono un assistente che stima il rischio clinico in base ad alcuni dati di base. ⚠️ Non sono uno strumento diagnostico.

Sei uomo o donna? (rispondi 'uomo' o 'donna')

uomo

Quanti anni hai?

centoventi

Inserisci un'età valida (un numero tra 1 e 120).

63

Che tipo di dolore al petto avverti?
1) angina tipica · 2) angina atipica · 3) dolore non anginoso · 4) asintomatico

1

Qual è la tua pressione arteriosa a riposo (mmHg)?

**Note:** Mostro il dialogo reale, compreso un turno sbagliato: è il comportamento che dimostra la robustezza dell'agente. Faccio notare che l'errore è specifico della domanda, non generico — ogni slot sa spiegare cosa si aspetta. Le parole "nuovo", "ricomincia" o "/start" reimpostano la sessione in qualsiasi momento.

# Come si arriva alla previsione

01 **Risposte raccolte** — il dizionario `answers` contiene un valore per ogni feature

02 **Record → testo** — `record_to_text()` compone una frase clinica in linguaggio naturale

03 **Testo → vettore** — BioBERT produce un embedding di 768 dimensioni

04 **Vettore → probabilità** — `predict_proba()` restituisce P(classe positiva)

05 **Probabilità → etichetta** — confronto con la soglia $\tau$ appresa

06 **Risposta** — esito, probabilità, soglia e avvertenza esplicita

OUTPUT REALE DELL'AGENTE

In base ai dati forniti, il paziente risulta **a RISCHIO di malattia cardiaca**.

Probabilità stimata: **78.4%** (soglia di decisione: 39.0%).

⚠️ Questo è solo un supporto informativo basato su un modello statistico, non una diagnosi medica. Consulta sempre un professionista sanitario.

Scrivi 'nuovo' per fare un'altra valutazione.

**Note:** Questo è il cuore della domanda "come dalle risposte si arriva alla predizione". Spiego che i passi 2 e 3 sono esattamente gli stessi usati in addestramento: la stessa funzione, lo stesso modello di embedding. È ciò che garantisce che il vettore prodotto in chat sia confrontabile con quelli su cui il classificatore è stato addestrato.

# Perché trasformare i dati in una frase

Il modello di embedding è stato addestrato su **testo biomedico**. Dandogli una frase invece di una riga di numeri, sfruttiamo la conoscenza linguistica che ha già.

### Cosa guadagniamo

Le etichette diventano parole che il modello ha già visto in letteratura medica — "angina tipica", "difetto reversibile" — invece di codici numerici (1, 2, 3) che da soli non significano nulla.

### Un solo punto di verità

`record_to_text()` è usata sia per addestrare sia per rispondere in chat. Addestramento e inferenza non possono divergere.

LA FRASE REALMENTE GENERATA E DATA IN PASTO AL MODELLO

```
Male patient, 63 years old,
chest pain type: angina tipica,
resting blood pressure: 145 mm Hg,
serum cholesterol: 233 mg/dl,
fasting blood sugar > 120 mg/dl: yes,
resting electrocardiographic results: normale,
maximum heart rate achieved: 150,
exercise-induced angina: no,
ST depression induced by exercise: 2.3,
slope of the peak exercise ST segment: discendente,
number of major vessels colored by fluoroscopy: 0,
thalassemia: difetto fisso
```

**Note:** Questa scelta è la più originale del progetto e vale la pena difenderla: trattiamo un problema tabellare come un problema di testo. Se mi chiedono perché non usare direttamente i numeri con un classificatore classico, rispondo onestamente: è un'alternativa legittima e spesso competitiva; qui l'obiettivo era verificare quanto segnale clinico un encoder biomedico riesce a catturare da una descrizione testuale.

# Gli embedding semantici

Un **embedding** è un vettore numerico che rappresenta il significato di un testo: frasi con contenuto simile producono vettori vicini nello spazio.

**768**
DIMENSIONI

**1**
VETTORE PER PAZIENTE

MODELLO USATO

**`pritamdeka/BioBERT-mnli-snli-scinli-scitail-mednli-stsb`**

Un **sentence-transformer** costruito su **BioBERT**: BERT pre-addestrato su letteratura biomedica (PubMed) e poi specializzato nel produrre vettori di frase confrontabili tra loro.

### Perché questo e non un modello generalista

In un confronto preliminare tra encoder generalisti e biomedici su entrambi i dataset, è risultato il migliore per **ROC-AUC**. Da qui la scelta di fissarlo come unico encoder del sistema (`EMBEDDING_MODEL` in `function.py`).

---

**Note:** Spiego l'embedding con l'intuizione geometrica: significati vicini, vettori vicini. Il dettaglio da sottolineare è che il modello è *congelato*: non lo riaddestriamo, lo usiamo come estrattore di caratteristiche. Tutto l'apprendimento specifico del compito avviene nello strato successivo, il classificatore.

# La classificazione

### Regressione logistica

Combina linearmente i 768 valori dell'embedding e passa il risultato in una **funzione sigmoide**, che lo comprime in una probabilità tra 0 e 1. In codice: `LogisticRegression(max_iter=2000)`.

### Perché un modello lineare e non una rete profonda

L'embedding ha già fatto il lavoro semantico pesante. Uno strato lineare su rappresentazioni congelate misura **quanto segnale è già presente** nel vettore, con molti meno parametri da stimare — decisivo su Heart Disease, dove i record sono poche centinaia.

### La soglia di decisione τ

Non si usa 0.5. La soglia viene **appresa**: è quella che massimizza l'F1 sulla curva precision-recall, calcolata su ogni fold e poi mediata.

**0.390**
T · HEART DISEASE

**0.376**
T · DIABETES 130

### Cosa significa una soglia bassa

Il sistema segnala "a rischio" anche quando la probabilità stimata è sotto il 50%: è **deliberatamente prudente**, perché in ambito clinico un falso negativo pesa più di un falso allarme.

**Note:** Se mi chiedono perché non una rete neurale: rispondo che con 814 record e 768 dimensioni avrei più parametri che esempi. La soglia è il punto più interessante da raccontare — mostra che la decisione finale non è "probabilità > 50%" ma una scelta calibrata sui dati, e che la calibrazione riflette un'asimmetria di costo tra i due tipi di errore.

# Come il sistema impara, prima della conversazione

01 **preprocessing.py** — carica i dati grezzi, imputa i valori mancanti, divide 80/20 e bilancia le classi con SMOTENC → restituisce `X, y`

02 **embedding.py** — converte ogni record in frase e genera i vettori BioBERT → salva `.npy` in `datas/<dataset>/embeddings/`

03 **classification.py** — validazione incrociata a 5 fold, calcolo delle metriche e della soglia → `model_performance.csv`

04 **chatbot_core.py** — riaddestra il classificatore su *tutti* gli embedding e carica la soglia salvata → l'agente è pronto

### Ogni avvio riparte da zero

`clean_output_dirs()` svuota le cartelle del dataset prima di ricostruire: nessun risultato di una esecuzione precedente può sopravvivere e mescolarsi con quella corrente.

### Separazione netta dei due momenti

Le fasi 1–3 sono **apprendimento** e avvengono una volta sola, all'avvio. La fase 4 prepara l'**inferenza**, che poi si ripete a ogni conversazione senza ulteriore addestramento.

**Note:** Qui rispondo alla domanda "come avviene l'apprendimento". Il messaggio da far passare: l'addestramento è un processo batch che precede la conversazione, non qualcosa che accade mentre l'utente scrive. Se mi chiedono i tempi: su Heart Disease è questione di minuti, su Diabetes130 molto di più perché gli embedding da generare sono 28.428 invece di 814.

# Preparazione dei dati e bilanciamento

PASSO 1

### Imputazione

I valori mancanti vengono riempiti con la **mediana** (feature numeriche) o la **moda** (categoriali). Necessario perché SMOTENC e la generazione del testo non tollerano buchi.

PASSO 2

### Split stratificato 80/20

La divisione conserva la proporzione delle classi, così la parte di addestramento non risulta sbilanciata per caso.

PASSO 3

### SMOTENC

Genera record sintetici della classe minoritaria fino a pareggiare le due classi.

### Perché SMOTENC e non SMOTE

SMOTE classico lavora in uno spazio numerico codificato: i record che genera non sono più traducibili in una frase sensata. **SMOTENC** gestisce insieme colonne numeriche e categoriali — interpola i numeri, assegna alle categorie il valore più frequente tra i vicini — quindi ogni record sintetico resta un **record clinico valido** e convertibile in testo esattamente come uno reale.

### Risultato del bilanciamento

| | | |
|---|---|---|
| **Heart Disease** | 407 / 407 | 814 |
| **Diabetes 130** | 14.214 / 14.214 | 28.428 |

**Note:** Il punto da spiegare bene è perché SMOTENC: è l'unica scelta compatibile con l'idea di convertire i record in testo. Se usassi SMOTE normale otterrei vettori sintetici che non corrispondono a nessun paziente descrivibile a parole. Se mi chiedono i rischi del bilanciamento: sì, le classi non sono più nelle proporzioni reali, e questo va tenuto presente quando si leggono accuratezza e F1.

# Validazione e metriche

### Validazione incrociata stratificata

`StratifiedKFold(n_splits=5, shuffle=True, random_state=42)`: i dati vengono divisi in 5 parti che conservano la proporzione delle classi. A turno, 4 servono per addestrare e 1 per valutare.

### Le tre metriche

**Accuracy** — quota di previsioni corrette. Leggibile, ma sensibile allo sbilanciamento.
**Macro-F1** — media armonica di precisione e recall calcolata su entrambe le classi con lo stesso peso.
**ROC-AUC** — capacità di *ordinare* i pazienti per rischio, indipendente dalla soglia scelta.

### Cosa succede in ogni fold

Addestra → stima le probabilità sulla parte tenuta fuori → calcola la curva precision-recall → sceglie la soglia che massimizza F1 → misura le metriche. I valori riportati sono la **media dei 5 fold**.

### Perché la ROC-AUC è la metrica di riferimento

È l'unica che non dipende da τ: misura la qualità del modello separatamente dalla politica di decisione. Per questo è stata usata per scegliere l'encoder.

---

**Note:** Spiego perché non basta una sola divisione train/test: con 814 record il risultato dipenderebbe troppo da quali record finiscono dove. La 5-fold usa tutti i dati sia per addestrare sia per valutare, in momenti diversi. Preparo la risposta alla possibile obiezione: la soglia è scelta sullo stesso fold su cui si misurano accuratezza e F1, quindi quei due numeri sono leggermente ottimistici; la ROC-AUC invece non ne risente.

# ❤️ UCI Heart Disease

**Compito: diagnosi.** Stabilire se è presente una malattia coronarica, sulla base dei dati di una singola visita cardiologica.

### Origine — quattro centri clinici

| | |
|---|---|
| Cleveland Clinic Foundation | 303 |
| Hungarian Institute of Cardiology | 294 |
| University Hospital, Zurigo/Basilea | 123 |
| V.A. Medical Center, Long Beach | 200 |
| **Totale concatenato** | **920** |

### Pulizia specifica

In alcuni centri colesterolo e pressione mancanti sono registrati come `0`: valori fisiologicamente impossibili, quindi trattati come dati mancanti e imputati.

### 13 feature cliniche

età · sesso · tipo di dolore toracico · pressione a riposo · colesterolo · glicemia a digiuno · ECG a riposo · frequenza cardiaca massima · angina da sforzo · depressione del tratto ST · pendenza ST · vasi colorati da fluoroscopia · talassemia

### Variabile target

`num > 0` → presenza di malattia. Il valore originale indica anche la gravità (1–4), ma viene **binarizzato**: presente / assente.

**920**
RECORD GREZZI

**814**
DOPO BILANCIAMENTO

**Note:** Da sottolineare: i quattro centri hanno qualità dei dati molto diversa — Cleveland è quasi completo, gli altri hanno moltissimi valori mancanti su alcune colonne (vasi colorati e talassemia in particolare). È un limite reale del dataset, non del codice, e lo dichiaro apertamente nella slide dei limiti.

# 🏥 UCI Diabetes 130-US Hospitals

**Compito: prognosi.** Prevedere se un paziente diabetico verrà **riammesso in ospedale entro 30 giorni** dalla dimissione.

### Origine e scala

**101.766** ricoveri di pazienti diabetici in **130 ospedali statunitensi**, raccolti tra il **1999 e il 2008**. Il progetto ne usa un **campione stratificato di 20.000** record, per contenere i tempi di calcolo mantenendo la proporzione delle classi.

### Selezione delle colonne: 19 su circa 50

Escluse quelle quasi costanti (flag di dosaggio di farmaci rari) e quelle con troppi valori mancanti o prive di significato clinico (peso, codice assicurativo, identificativi del paziente).

### 19 feature

etnia · genere · fascia d'età · tipo di ammissione · modalità di dimissione · fonte di ammissione · giorni di degenza · numero di esami di laboratorio · procedure · farmaci somministrati · visite ambulatoriali, accessi al pronto soccorso e ricoveri nell'anno precedente · numero di diagnosi · glicemia massima · test A1C · terapia insulinica · cambio di terapia · farmaco antidiabetico prescritto

### Variabile target

`readmitted == "<30"` → riammissione precoce. Le riammissioni oltre i 30 giorni e le non riammissioni contano entrambe come classe negativa.

**20k**
CAMPIONE USATO

**28.428**
DOPO BILANCIAMENTO

**Note:** Il contrasto da evidenziare: qui il compito è prognostico, molto più difficile, e le feature sono in gran parte amministrative (conteggi, codici di ammissione) più che cliniche. Questo spiega in buona parte perché le metriche saranno più basse: le informazioni che servirebbero davvero — aderenza alla terapia, supporto familiare, condizioni sociali — non sono nel dataset.

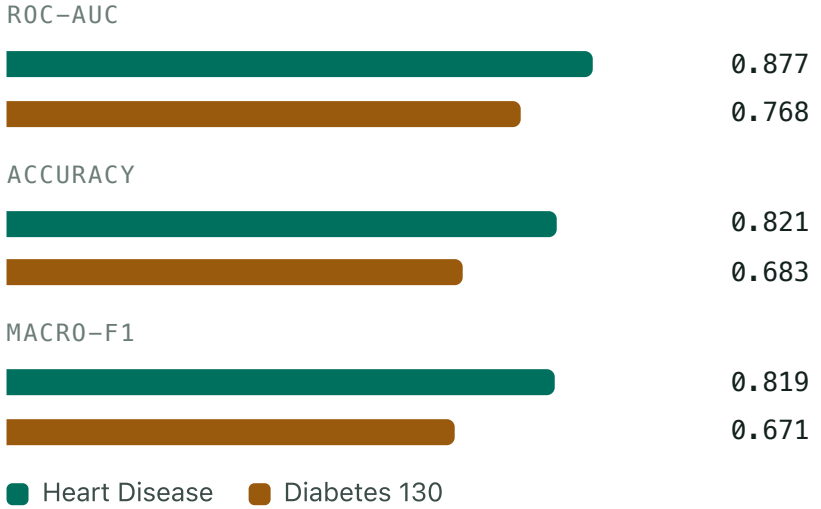
# Confronto e criteri di scelta

| CRITERIO | HEART DISEASE | DIABETES 130 |
|---|---|---|
| Compito | **Diagnosi** | **Prognosi a 30 giorni** |
| Record grezzi | 920 | 101.766 |
| Usati per addestrare | 814 | 28.428 |
| Feature | 13 | 19 |
| Domande in chat | 13 | 19 |
| Natura dei dati | Cliniche / strumentali | Prevalentemente amministrative |
| Tempo di avvio | Minuti | Molto più lungo |

### Come scegliere

**Cosa vuoi mostrare** — diagnosi o previsione di riammissione · **Chi è l'utente** — Heart Disease chiede esami specialistici (ECG da sforzo, fluoroscopia), Diabetes 130 chiede dati che conosce il personale ospedaliero · **Quanto tempo hai** — 814 contro 28.428 embedding da generare.

RISULTATI · MEDIA SU 5 FOLD




### Per la demo d'esame

**Heart Disease**: si avvia in pochi minuti, le domande sono riconoscibilmente cliniche e le metriche sono nettamente migliori.

**Note:** Se mi chiedono perché Diabetes 130 va peggio, la risposta è nella natura del compito: prevedere una riammissione futura dipende da fattori che il dataset non contiene. Non è un difetto dell'implementazione. Faccio notare che il divario è coerente su tutte e tre le metriche, quindi non è un artefatto di una misura particolare.

# Come si esegue il progetto

### Prerequisiti

Python **3.11+** · connessione a internet al primo avvio (il modello BioBERT viene scaricato automaticamente) · un token **Telegram** solo se si usa quel front-end.

OPZIONE A · SCRIPT AUTOMATICO

```
# crea l'ambiente, installa, chiede cosa lanciare
./run_chatbot.sh

# oppure direttamente con i parametri
./run_chatbot.sh --dataset heart_disease \
                 --chatbot streamlit
```

OPZIONE B · MANUALE

```
python3 -m venv env
source env/bin/activate
pip install -r requirements.txt

python main.py --dataset heart_disease \
               --chatbot streamlit

python main.py --dataset diabetes130 \
               --chatbot telegram
```

### Cosa succede a ogni avvio

`main.py` **ricostruisce il classificatore da zero** — pulizia, preprocessing, embedding, addestramento — e solo quando è pronto apre il chatbot. Con `diabetes130` l'attesa è sensibilmente più lunga: gli embedding da calcolare sono 28.428.

### Token Telegram

File `.env` nella cartella del progetto:
`TELEGRAM_BOT_TOKEN=...` ottenuto da @BotFather.

**Note:** Durante la demo conviene avviare il progetto *prima* di iniziare a parlare, perché la fase di embedding richiede tempo. Se la connessione non è disponibile, il modello BioBERT deve essere già in cache locale dal primo avvio. Consiglio: usare Streamlit in aula, perché non richiede token né connessione a Telegram.

# I due front-end

APP_STREAMLIT.PY

### Interfaccia web locale

Usa `st.chat_message` e `st.chat_input` per la chat; lo stato della conversazione vive in `st.session_state`, che sopravvive ai ri-render della pagina.

Se il dataset non è preimpostato da riga di comando, lo si sceglie direttamente nell'interfaccia prima di iniziare.

BOT_TELEGRAM.PY

### Bot Telegram asincrono

Costruito con `Application` e due handler: `/start` e i messaggi di testo. Funziona in **polling**.

Mantiene un dizionario `sessions` indicizzato per `chat_id`: **più utenti possono conversare contemporaneamente**, ognuno con il proprio stato.

### Un'unica logica, due canali

Entrambi i front-end si limitano a ricevere il testo e passarlo a `chatbot_core.handle_message()`, poi mostrano la risposta. Nessuna delle due interfacce contiene logica di dialogo o di predizione: cambiare canale non cambia il comportamento dell'agente. È la stessa separazione che, nella teoria degli agenti, distingue l'**agente** dai suoi **sensori e attuatori**.

---

**Note:** Questo è un buon punto da far valere all'esame: la separazione tra logica dell'agente e canale di interazione è esattamente il principio architetturale visto a lezione. La prova pratica è che il bot Telegram gestisce più conversazioni simultanee senza che il core sappia nulla di Telegram.

# Le librerie e il loro ruolo

| LIBRERIA | VERSIONE | RUOLO NEL PROGETTO |
|---|---|---|
| **sentence-transformers** | `5.5.1` | Carica BioBERT e trasforma la frase clinica in un vettore di 768 numeri ( `model.encode` ). È il ponte tra testo e matematica. |
| **transformers** + **torch** | `5.7.0 / 2.11.0` | Motore sottostante del modello neurale: tokenizzazione del testo ed esecuzione della rete. |
| **scikit-learn** | `1.8.0` | Regressione logistica, validazione incrociata stratificata, metriche e curva precision-recall per la soglia $\tau$. |
| **imbalanced-learn** | `0.14.2` | SMOTENC: bilancia le classi generando record sintetici che restano clinicamente interpretabili. |
| **pandas** | `3.0.2` | Lettura dei file `.data` e `.csv` , selezione delle colonne, imputazione dei mancanti. |
| **numpy** | `2.4.4` | Gestione e salvataggio degli array di embedding in formato `.npy` . |
| **streamlit** | `1.59.1` | Interfaccia chat web e persistenza dello stato della sessione. |
| **python-telegram-bot** | `22.8` | Bot Telegram asincrono: gestione degli handler e del polling dei messaggi. |
| **python-dotenv** | `1.2.2` | Carica il token Telegram da `.env` , tenendolo fuori dal codice sorgente. |

**Note:** Non leggo la tabella riga per riga. Mi concentro sulle tre che fanno il lavoro concettuale: sentence-transformers (il testo diventa vettore), scikit-learn (il vettore diventa probabilità), imbalanced-learn (i dati diventano bilanciati senza perdere interpretabilità). Le altre sono infrastruttura.

# Limiti dichiarati e sviluppi futuri

### Non è uno strumento diagnostico

Il codice lo dichiara due volte: nel messaggio di benvenuto e dopo ogni previsione. È un supporto informativo basato su un modello statistico.

### Asimmetria tra addestramento e uso

Nel training molti valori erano imputati; in chat l'utente fornisce sempre un valore reale. Le due situazioni non sono identiche.

### Soglia e classificatore non perfettamente allineati

$\tau$ è la media delle soglie dei 5 fold, ma il classificatore finale viene riaddestrato su tutti i dati: non è esattamente lo stesso modello su cui $\tau$ è stata calibrata.

### Dati storici e non italiani

Heart Disease risale al 1988, Diabetes 130 copre il 1999–2008, entrambi su popolazioni statunitensi ed europee.

### Sviluppi possibili

**Permettere "non lo so"** — oggi ogni domanda richiede una risposta, anche quando l'utente non può conoscerla (fluoroscopia, ECG da sforzo).

**Calibrare le probabilità** — verificare che un "70%" corrisponda davvero a 7 casi su 10, non solo che l'ordinamento sia corretto.

**Spiegare la previsione** — indicare quali risposte hanno pesato di più sull'esito.

**Validare lo scenario conversazionale** — le metriche misurano il modello sui dati, non l'affidabilità del sistema usato da una persona reale.

### In sintesi

Un agente conversazionale **task-oriented** che unisce un dialogo deterministico e verificabile a una decisione appresa dai dati, con le sue prestazioni **misurate** e i suoi limiti **dichiarati**.

**Note:** Chiudo con i limiti, non con i risultati: dimostra consapevolezza critica ed è ciò che normalmente viene chiesto in sede d'esame. Il limite più interessante da discutere è il primo degli sviluppi: il chatbot obbliga a rispondere a tutto, mentre il modello sarebbe perfettamente in grado di gestire un valore dichiarato come non disponibile. Grazie, sono a disposizione per le domande.